

# Problems with RNNs: Long-Term Dependency and Unstable Gradients

## Contents

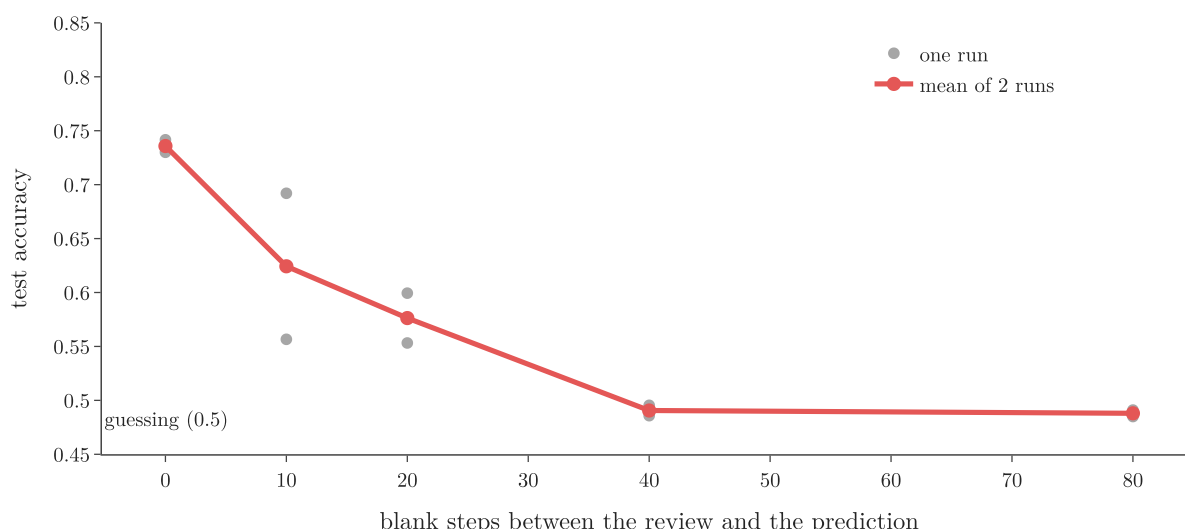
|                                                        |    |
|--------------------------------------------------------|----|
| 1. Overview . . . . .                                  | 2  |
| 2. Prerequisites . . . . .                             | 2  |
| 3. The long-term dependency problem . . . . .          | 3  |
| 3.1 Short-term memory . . . . .                        | 3  |
| 3.2 An example: predicting the next word . . . . .     | 3  |
| 3.3 The problem on real reviews . . . . .              | 4  |
| 4. Why: the vanishing gradient through time . . . . .  | 4  |
| 4.1 Short-term and long-term terms . . . . .           | 5  |
| 4.2 With 100 time steps . . . . .                      | 5  |
| 4.3 One factor of the chain . . . . .                  | 5  |
| 4.4 What is special about an RNN . . . . .             | 6  |
| 4.5 The trained RNN on real reviews . . . . .          | 7  |
| 5. Fixing the vanishing gradient . . . . .             | 8  |
| 6. Unstable training: the exploding gradient . . . . . | 9  |
| 6.1 How the gradient explodes . . . . .                | 9  |
| 6.2 Fixing the exploding gradient . . . . .            | 10 |
| 7. Summary . . . . .                                   | 11 |
| 8. Sources . . . . .                                   | 11 |
| 9. Key terms . . . . .                                 | 11 |

# 1. Overview

**Key point:** A simple RNN has two problems: it cannot learn long-term dependencies, and its training can become unstable. Both come from one fact: during backpropagation through time the gradient is multiplied by the same recurrent weights again and again, so it shrinks (vanishes) or grows (explodes) with distance.

RNNs suit **sequential data**: text, time series, any data where a point depends on the points before it. Yet plain RNNs are rarely used in practice. They suffer from two problems:

1. **The long-term dependency problem**, caused by the vanishing gradient.
2. **Unstable training**, where the network fails to train, caused by the exploding gradient.



**Figure 1:** Test accuracy of a SimpleRNN on IMDB reviews of 50 words followed by  $G$  blank time steps before the prediction. Grey dots: single runs; red: the mean of 2 runs. The longer the gap the RNN must remember across, the closer it gets to guessing

Figure 1 shows the first problem on real data. The review is the same in every setting; only the distance between the review and the prediction grows, and the RNN loses the review's sentiment along the way. This Note explains why, and what can be done about it; the cure most used in practice, the LSTM, has its own Notes.

## 2. Prerequisites

- The [backpropagation through time Note](#): the gradient of  $W_i$  and  $W_h$  is a sum of one term per time step.
- The [vanishing and exploding gradients Note](#): a long product of small factors vanishes, of large factors explodes; gradient clipping.
- The [activation functions Note](#): the slope of  $\tanh$  is between 0 and 1.
- The [RNN forward propagation Note](#):  $h_t = \tanh(x_t W_i + h_{t-1} W_h)$ .

### 3. The long-term dependency problem

**Key point:** When an output depends on an input many time steps earlier, a simple RNN fails: it remembers recent inputs and forgets old ones.

#### 3.1 Short-term memory

**Key point:** As the number of time steps grows, the RNN cannot keep information from the early steps.

In sequential data, later items depend on earlier ones. When the sequence is short, an RNN handles this well. When the sequence becomes long, with many time steps, the RNN cannot remember the early steps by the time it reaches the late ones. A simple RNN behaves like a person with short-term memory loss: recent events are clear, older ones are gone.

#### 3.2 An example: predicting the next word

**Key point:** “Marathi is spoken in \_\_\_\_” needs a word 4 steps back; the long sentence of Figure 2 needs one 22 steps back. A simple RNN handles the first and fails at the second.

Take next-word prediction, as on a phone keyboard.

##### short-term dependency

Marathi is spoken in Maharashtra

4 time steps back

##### long-term dependency

Maharashtra is a beautiful place. I went there last year, but I could not enjoy it properly because I do not understand Marathi

22 time steps back

**Figure 2:** Two next-word predictions. The short sentence needs a word 4 time steps back; the long one needs a word 22 time steps back

- In “Marathi is spoken in Maharashtra”, the word Maharashtra depends on Marathi, 4 time steps back. The dependency is **short-term**.

- In “Maharashtra is a beautiful place. I went there last year, but I could not enjoy it properly because I do not understand \_\_\_\_\_”, the missing word, Marathi, depends on Maharashtra, 22 time steps back. The dependency is **long-term**.

A keyboard built on a simple RNN performs well on sentences like the first and poorly on sentences like the second. This failure is the **long-term dependency problem**: an output at one time step depends on an input from a much earlier time step, and the RNN does not have the memory to connect them.

Bengio, Simard and Frasconi (1994) showed why gradient-based learning faces an increasingly difficult problem as the duration of the dependencies grows. Goodfellow §10.7 summarises: the probability of successfully training a traditional RNN with stochastic gradient descent rapidly reaches 0 for sequences of only length 10 or 20.

### 3.3 The problem on real reviews

**Key point:** The same SimpleRNN, the same reviews; only a gap of blank steps between the review and the prediction changes. Test accuracy falls from 0.736 with no gap to 0.488 with a gap of 80 steps: guessing level.

We take IMDB movie reviews: each **observation** (one record) is a review, and the **target** (the output we predict) is its sentiment, positive or negative. Each review is cut to its last 50 words. Then  $G$  blank time steps (the padding token 0) are added after it, before the network gives its prediction. The network now has to carry the sentiment of the review across  $G$  steps of nothing.

Each setting is trained on 10,000 reviews for 5 epochs and tested on 5,000, twice with different seeds (Notebook). Figure 1 and the table give the mean test accuracy:

| Gap $G$ (blank steps) | 0     | 10    | 20    | 40    | 80    |
|-----------------------|-------|-------|-------|-------|-------|
| Test accuracy         | 0.736 | 0.624 | 0.576 | 0.491 | 0.488 |

The information is in the data in every case: only the distance changes. The RNN cannot learn to carry it over a long distance. The next section shows why.

## 4. Why: the vanishing gradient through time

**Key point:** The gradient of  $W_i$  is a sum of short-term terms (recent inputs) and long-term terms (distant inputs). Each long-term term contains a long product of factors  $\partial h_t / \partial h_{t-1}$ , which shrinks towards 0. So the weights learn almost only from recent inputs.

## 4.1 Short-term and long-term terms

**Key point:** In the BPTT sum, the term for the last input is short; the term for the first input is the longest chain.

Take a single hidden node, so every weight is one number, and three time steps: inputs  $x_1, x_2, x_3$ , hidden states  $h_1, h_2, h_3$ , prediction  $\hat{y}$ . From the [backpropagation through time](#) [Note](#), the gradient of the input weight is a sum of three terms, one for each use of  $w_i$ :

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_i} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_i}$$

- The **first** term measures how the loss changes through the **last** input,  $x_3$ : a **short-term** contribution.
- The **last** term measures how the loss changes through the **first** input,  $x_1$ , the one farthest from the output: a **long-term** contribution.

Pascanu, Mikolov and Bengio (2013) call these the temporal contributions of the gradient and use the same split into long-term and short-term ones.

## 4.2 With 100 time steps

**Key point:** The long-term term carries a product of 99 factors  $\partial h_t / \partial h_{t-1}$ .

Real sequences have 100 time steps or more. The longest term then reads

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \frac{\partial h_{100}}{\partial h_{99}} \frac{\partial h_{99}}{\partial h_{98}} \dots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_i}$$

the second-longest stops at  $h_2$ , the third at  $h_3$ , and so on. We can write the long chain compactly as a product:

$$\frac{\partial h_{100}}{\partial h_{99}} \dots \frac{\partial h_2}{\partial h_1} = \prod_{t=2}^{100} \frac{\partial h_t}{\partial h_{t-1}}$$

## 4.3 One factor of the chain

**Key point:**  $\partial h_t / \partial h_{t-1} = \tanh'(\cdot) w_h$ : the slope of tanh times the feedback weight.

1. **In words:**  $h_t$  is tanh of something that contains  $h_{t-1} w_h$ . Differentiating with respect to  $h_{t-1}$  gives the slope of tanh at that point times  $w_h$ .
2. **Formula:**

$$h_t = \tanh(x_t w_i + h_{t-1} w_h) \quad \Rightarrow \quad \frac{\partial h_t}{\partial h_{t-1}} = \tanh'(x_t w_i + h_{t-1} w_h) w_h$$

so the long-term term becomes

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \left( \prod_{t=2}^{100} \tanh'(\cdot) w_h \right) \frac{\partial h_1}{\partial w_i}$$

3. **Example:** the slope of tanh is between 0 and 1 (see the [activation functions Note](#)). Suppose every slope is 0.8 and  $w_h = 0.9$ . Each factor is  $0.8 \times 0.9 = 0.72$ , and 99 of them give

$$0.72^{99} \approx 7.5 \times 10^{-15}$$

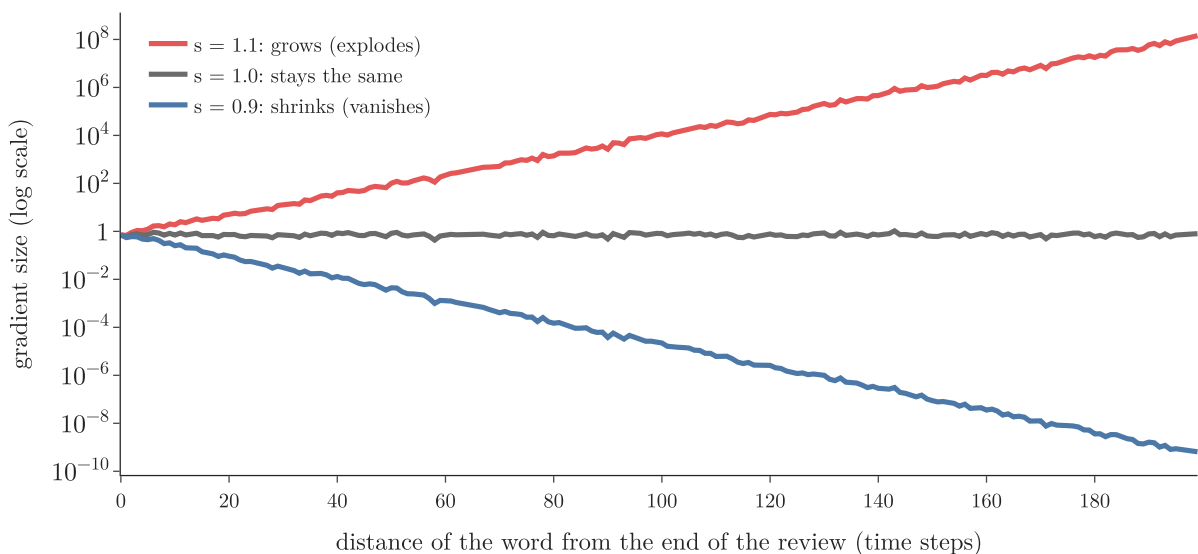
The long-term term is practically 0. Its whole contribution to the gradient disappears, and the gradient is made almost entirely of short-term terms. The weights are therefore updated to fit the **recent** inputs, and the influence of distant inputs is not learned. That is the long-term dependency problem, and it is a **vanishing gradient problem** (see the [vanishing gradients Note](#)): the farther the input, the smaller its gradient.

**Extra:** With several hidden nodes, each factor is a matrix, the Jacobian  $\partial h_t / \partial h_{t-1}$ . In the row-vector form of the [forward propagation Note](#) it is  $W_h$  with each column scaled by that node's tanh slope; Goodfellow (§10.2.2, eq. 10.21) writes the same thing as  $W^T \text{diag}(1 - h^2)$ . Pascanu et al. (2013, §2.1) prove that if the largest absolute eigenvalue of the recurrent weight matrix is below  $1/\gamma$ , where  $\gamma$  bounds the slope of the activation ( $\gamma = 1$  for tanh), the long-term contributions vanish as the distance grows.

#### 4.4 What is special about an RNN

**Key point:** A deep ANN multiplies **different** weights layer after layer; an RNN multiplies the **same**  $W_h$  at every step. A number below 1 raised to a power always vanishes, and one above 1 always explodes.

The [vanishing gradients Note](#) met long products in deep ANNs. An RNN is worse in one specific way: the factor is the same at every step. Goodfellow §10.7 calls this problem particular to recurrent networks. Leave out tanh and the inputs, and the recurrence is  $h_t = h_{t-1} W_h$ , so after  $t$  steps the state, and the gradient sent back, is multiplied by  $W_h$  to the power  $t$ . The eigenvalues of  $W_h$  are raised to the power  $t$ : those with magnitude below 1 decay to 0, those above 1 explode (Goodfellow §10.7, eq. 10.36 to 10.39). With one node this is just  $w_h^t$ . A deep ANN with a different random weight at each layer can be scaled to avoid the problem; the shared weight of an RNN cannot (Goodfellow §10.7).



**Figure 3:** Gradient size by distance in an untrained SimpleRNN without activation on IMDB reviews, with  $W_h$  set to  $s$  times an orthogonal matrix (all eigenvalues of magnitude 1). Each step multiplies the gradient by about  $s$

The Notebook tests this on real reviews. A SimpleRNN with no activation gets  $W_h = sQ$ , where  $Q$  is an orthogonal matrix (every eigenvalue has magnitude exactly 1), so every eigenvalue of  $W_h$  has magnitude  $s$ . Figure 3 shows the result:

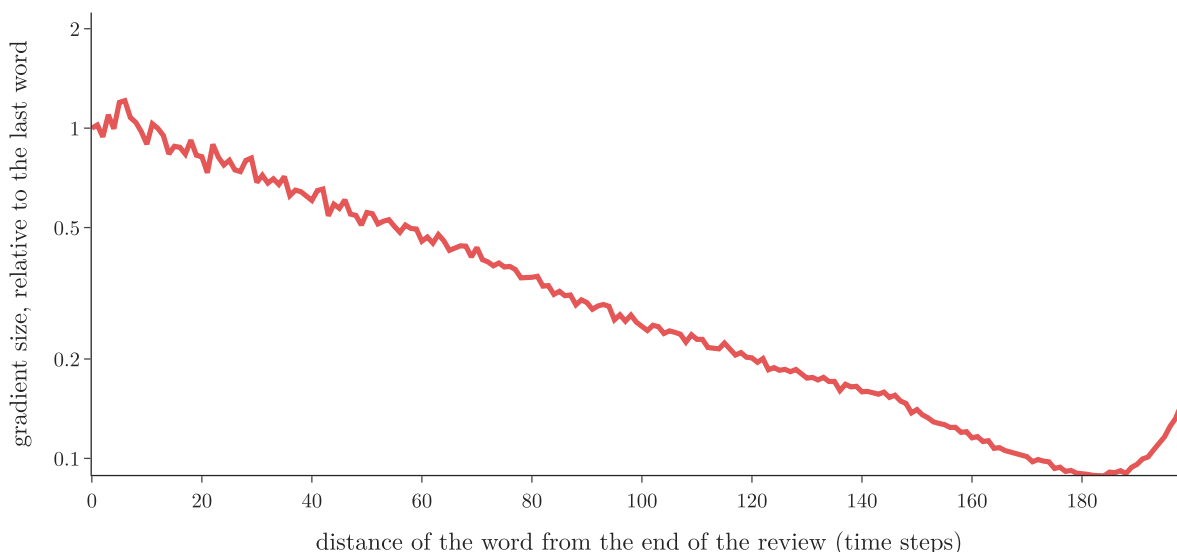
| $s$ | Gradient 50 steps back, relative to the last word | $s^{50}$ |
|-----|---------------------------------------------------|----------|
| 0.9 | 0.0062                                            | 0.0052   |
| 1.0 | 1.2                                               | 1        |
| 1.1 | 141                                               | 117      |

The measured ratios follow  $s^{50}$ : only the size of the shared weight decides whether the gradient vanishes, stays or explodes.

#### 4.5 The trained RNN on real reviews

**Key point:** In a tanh SimpleRNN trained on IMDB, a word 50 steps from the end moves the loss about half as much as the last word, and a word 100 steps back a quarter as much. The decay is gradual because Keras starts  $W_h$  as an orthogonal matrix.

We train a SimpleRNN with 32 nodes on 10,000 IMDB reviews (their last 200 words) for 3 epochs; it reaches 0.795 test accuracy. For 500 test reviews of at least 200 words, the Notebook measures, with `tf.GradientTape`, the size of the gradient of the loss with respect to each word's vector.



**Figure 4:** Gradient of the loss with respect to each word of a review, relative to the last word, for a SimpleRNN trained on IMDB (mean over 500 reviews of 200 words). Log scale

| Distance (steps)  | 0 | 10   | 20   | 50   | 100  | 199  |
|-------------------|---|------|------|------|------|------|
| Relative gradient | 1 | 0.89 | 0.82 | 0.56 | 0.25 | 0.15 |

Figure 4 shows the gradient shrinking steadily with distance: the words far from the end send the weights a much weaker training signal than the last words.

The decay is slower than the  $0.72^{99}$  of section 4.3. Keras starts  $W_h$  as an orthogonal matrix (Keras documentation, `SimpleRNN`, `recurrent_initializer="orthogonal"`), so every eigenvalue of  $W_h$  has magnitude 1 at the start: the  $s = 1.0$  case of Figure 3, where  $W_h$  alone neither shrinks nor grows the gradient. The shrinking left over comes from the tanh slopes, which are below 1. Starting  $W_h$  well is one of the fixes of the next section, built into Keras.

## 5. Fixing the vanishing gradient

**Key point:** Four options: a different activation, a better initialisation of  $W_h$ , skip connections through time, or an LSTM. In practice the LSTM won.

1. **A different activation function.** The slope of tanh is between 0 and 1, which shrinks every factor. ReLU and Leaky ReLU (see the [ReLU variants Note](#)) have slope 1 for positive inputs, so they do not shrink the factor; the factor is then  $w_h$  alone.
2. **Better weight initialisation.** If  $W_h$  starts with factors below 1, the product vanishes. Starting  $W_h$  as the **identity matrix** (1 on the diagonal, 0 elsewhere) makes a multiplication by  $W_h$  leave the gradient unchanged at the start of training. Le, Jaitly and Hinton (2015) combined ReLU with an identity-initialised  $W_h$  and found it comparable to an LSTM on their four benchmarks.

3. **Skip connections through time.** Connections from a state several time steps back directly to the present give the gradient shorter paths. With a delay of  $d$  steps, gradients shrink with  $\tau/d$  instead of  $\tau$  steps (Goodfellow §10.9.1).
4. **Switch to an LSTM.** The LSTM was designed for exactly this problem, the “insufficient, decaying error backflow” of recurrent networks (Hochreiter and Schmidhuber 1997). Gated RNNs, the LSTM and the GRU, are the most effective sequence models used in practical applications (Goodfellow §10.10), so the usual choice is to move to one (see the [LSTM Note](#)).

**Python:** The first two fixes in Keras.

```
layer = keras.layers.SimpleRNN(
    32, activation="relu",
    recurrent_initializer="identity")
```

## 6. Unstable training: the exploding gradient

**Key point:** The opposite case: if the factors are above 1, the long-term terms grow huge, dominate the gradient and can reach infinity. The updates throw the weights away and the network does not train.

### 6.1 How the gradient explodes

**Key point:** If each factor  $\tanh'(\cdot) w_h$  is above 1, the product over many steps blows up.

Now the long-term terms become so large that they dominate the short-term ones, and can grow to infinity. The gradient update then becomes huge, the weights become huge or infinite, and the model does not train. The training stagnates: the loss stops improving.

1. **In words:** a factor above 1, multiplied by itself over many steps, grows without limit.
2. **Formula:** with ReLU, the slope is 1 for a positive input (and 0 for a negative one), so the factor is  $w_h$  and the product over  $t$  steps is  $w_h^t$ .
3. **Example:** with  $w_h = 1.1$  and 100 steps,  $1.1^{100} \approx 13,781$ ; with  $w_h = 1.5$ ,  $1.5^{100} \approx 4 \times 10^{17}$ .

Two situations make it likely:

- **ReLU with large recurrent weights.** ReLU does not squash: its slope is 1 for every positive input. If the recurrent weights are initialised large, nothing keeps the product small, and it explodes. The  $s = 1.1$  line of Figure 3 shows the same growth with no activation at all.
- **A high learning rate.** The gradient itself does not depend on the learning rate, but the step does:  $\Delta W = \eta \partial L / \partial W$ . A large gradient times a large  $\eta$  gives a huge step, which throws the weights far away, the case shown in section 7.1 of the [exploding gradients Note](#).

Goodfellow §10.7 notes that gradients over many steps vanish most of the time and explode rarely, but with much damage to the optimisation.

## 6.2 Fixing the exploding gradient

**Key point:** Gradient clipping, a moderate learning rate, or an LSTM.

1. **Gradient clipping.** Cap the size of the gradient: if its norm exceeds a threshold, scale it down to the threshold. Clipping is taught in the [exploding gradients Note](#); Pascanu et al. (2013, §3.2) proposed this norm clipping for RNNs, and Goodfellow §10.11.1 presents it as a fix for the cliffs that recurrent networks create in the loss.
2. **A controlled learning rate.** A smaller  $\eta$  keeps each step small even when the gradient is large.
3. **An LSTM.** The usual choice in practice, as for the vanishing gradient. The LSTM was built to stop the error from decaying (Hochreiter and Schmidhuber 1997); gradient clipping is still the standard guard against explosions in recurrent networks (Goodfellow §10.11.1).

**Python:** Clipping in Keras: every gradient is scaled down to norm at most 1 before the update.

```
opt = keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)
```

## 7. Summary

|                 | Long-term dependency                                            | Unstable training                                        |
|-----------------|-----------------------------------------------------------------|----------------------------------------------------------|
| Cause           | vanishing gradient through time                                 | exploding gradient through time                          |
| Factor per step | $\tanh'(\cdot) w_h$ below 1                                     | $\tanh'(\cdot) w_h$ above 1<br>(e.g. ReLU, large $w_h$ ) |
| Symptom         | only recent inputs influence learning                           | huge updates, loss stops improving or becomes NaN        |
| Fixes           | ReLU, identity initialisation of $W_h$ , skip connections, LSTM | gradient clipping, smaller learning rate, LSTM           |

- Both problems come from unstable gradients: in BPTT the long-term terms contain a product of many factors  $\partial h_t / \partial h_{t-1}$ .
- The factor is the same  $W_h$  at every step, so the product behaves like a power: below 1 it vanishes, above 1 it explodes.
- On IMDB, a SimpleRNN's accuracy falls from 0.736 to 0.488, guessing level, when 80 blank steps separate the review from the prediction; its gradient shrinks steadily with a word's distance from the end.
- Simple RNNs are therefore rarely used; LSTMs, which address these problems, replaced them.

## 8. Sources

- Bengio, Y., Simard, P. and Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks* 5(2), 157–166.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 10, §10.2.2 (gradient in an RNN), §10.7 (the challenge of long-term dependencies), §10.9.1 (skip connections through time), §10.10 (gated RNNs), §10.11.1 (clipping gradients). [deeplearningbook.org/contents/rnn.html](http://deeplearningbook.org/contents/rnn.html).
- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation* 9(8), 1735–1780.
- Le, Q. V., Jaitly, N. and Hinton, G. E. (2015). A Simple Way to Initialize Recurrent Networks of Rectified Linear Units. arXiv:1504.00941.
- Pascanu, R., Mikolov, T. and Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. ICML 2013. arXiv:1211.5063.
- Keras API documentation: SimpleRNN layer ([keras.io/api/layers/recurrent\\_layers/simple\\_rnn/](https://keras.io/api/layers/recurrent_layers/simple_rnn/)), Identity initializer, optimizer argument `clipnorm`.

## 9. Key terms

| Term                                   | Meaning                                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------------------|
| Observation                            | One record of the data, here one movie review                                                         |
| Target                                 | The output we predict, here the sentiment of a review                                                 |
| Long-term dependency                   | An output that depends on an input many time steps earlier                                            |
| Long-term dependency problem           | A simple RNN's failure to learn long-term dependencies, caused by the vanishing gradient through time |
| Short-term and long-term contributions | The terms of the BPTT gradient for recent and for distant inputs                                      |
| Unstable training                      | Training that does not progress because exploding gradients make the updates huge                     |
| Identity initialisation                | Starting the recurrent weight matrix as the identity matrix                                           |
| Skip connection through time           | A connection from a hidden state several steps back directly to the present                           |